

A Survey of Random Walk Variants on Graphs and Their Natural Lifts to Higher-Order Structures

Foundational notes for higher-order message passing and random-walk-based propagation

Adrián Ruiz Doblas

Abstract

Random walks are among the most basic stochastic processes on relational structures, and they sit at the interface of probability, spectral graph theory, diffusion geometry, and message passing. In graph learning, they are not merely analytical tools: they also define propagation operators, diffusion kernels, and implicit notions of neighborhood [6, 15]. This survey gives a self-contained overview of standard random walk variants on graphs and develops their natural lifts to higher-order structures, with emphasis on hypergraphs and, at a more schematic level, simplicial, cellular, and combinatorial complexes. We begin from basic probability and finite-state Markov-chain preliminaries, then review simple, lazy, weighted, restart-based, multi-step, continuous-time, and non-backtracking random walks on graphs. We then formulate a general operator-centric template for higher-order lifts and use it to compare several hypergraph walk constructions, including projected walks, node-hyperedge-node incidence walks, size-biased variants, restart and diffusion extensions, and non-backtracking lifts. We also explain how random-walk ideas reappear on simplicial and combinatorial-complex domains through Hodge-type operators, Hasse-graph constructions, and multi-neighborhood propagation schemes [7, 8, 12]. Throughout, the guiding perspective is that a choice of random walk is a choice of propagation geometry. This matters both for higher-order message passing and for structural questions such as role discovery, distinguishability, and regular/structural equivalence [5, 13]. The survey is intended as a mathematical foundation for the subsequent research step: comparing which hypergraph-native walk geometries best support higher-order message passing and role-sensitive computation.

Contents

1	Introduction and scope	3
1.1	Why this review?	3
1.2	What this document covers	4
1.3	Primary references and guiding question	4
2	Probability and Markov-chain preliminaries	5
2.1	Probability spaces, random variables, and stochastic processes	5
2.2	Finite-state distributions and stochastic matrices	6
2.3	Finite-state Markov chains	7
3	Graphs and the basic random walk picture	8
3.1	Graphs and matrices	8

3.2	Simple random walk	8
3.3	Random walks, Laplacians, and diffusion	9
3.4	Random walks as propagation rules in GNNs	10
4	Standard random walk variants on graphs	10
4.1	Lazy random walks	10
4.2	Weighted and biased walks	11
4.3	Random walk with restart and PageRank	11
4.4	Multi-step kernels and diffusion polynomials	12
4.5	Continuous-time random walks	12
4.6	Non-backtracking walks	13
4.7	Path-based versus walk-based neighborhoods	13
5	Higher-order domains and the operator viewpoint	13
5.1	Hypergraphs	13
5.2	Simplicial complexes	14
5.3	Cellular and combinatorial complexes	14
5.4	Random walks as propagation operators	14
6	A general template for hypergraph random walks	15
6.1	Node-hyperedge-node dynamics	15
6.2	Self-loops versus non-lazy conventions	16
7	Natural hypergraph lifts of graph random walk variants	16
7.1	Projected random walk	16
7.2	Uniform-incidence higher-order walk	16
7.3	Size-biased hyperedge selection	17
7.4	Microscopic versus projected viewpoints	18
7.5	Lazy hypergraph lifts	18
7.6	Restart and personalized higher-order diffusion	18
7.7	Multi-step hypergraph kernels	19
7.8	Non-backtracking hypergraph lifts	19
8	Natural lifts beyond hypergraphs	19
8.1	Simplicial complexes: up, down, and same-rank walks	19
8.2	Edge-based walks and Hodge operators	20
8.3	Across-dimension walks	21
8.4	Combinatorial complexes and multi-neighborhood lifts	21

9	Comparing walk variants: what actually changes?	21
9.1	State space	21
9.2	Neighborhood notion	22
9.3	Bias and stationary distribution	22
9.4	Query complexity and locality	22
9.5	One-step versus multi-step geometry	23
10	Structural roles, equivalence, and why the walk matters	23
10.1	Structural equivalence versus regular equivalence	23
10.2	Neighborhood graphs induced by walks	23
10.3	Curvature and transport viewpoint	24
11	Random walks as propagation operators in higher-order message passing	24
11.1	Graph case	24
11.2	Hypergraph case	24
11.3	Why restart and multi-step lifts are especially relevant	25
12	Synthesis and outlook	25
12.1	A useful list of design knobs	25
12.2	What counts as a natural lift?	26
12.3	Returning to the open question	26
13	Conclusion	26
A	A compact dictionary of graph and hypergraph operators	29
B	Minimal operator-centric template for later use	29
C	Derivation of the symmetric normalized Laplacian	30

1 Introduction and scope

1.1 Why this review?

A central lesson of graph learning is that the propagation rule is not a secondary implementation detail. In a message-passing architecture, one always chooses some notion of neighborhood and some operator that transports or mixes information through that neighborhood. On ordinary graphs, this choice is often hidden inside a normalized adjacency matrix, a Laplacian, or a diffusion kernel. Random walks provide a clean probabilistic language for this choice, and this operator viewpoint is already implicit in standard graph neural network formulations [6, 10, 15].

The present review is motivated by the next step of a broader project on higher-order message passing. The previous survey established the basic higher-order domains and the main message-passing frameworks used on them, and it left open a natural follow-up task: to review standard random walk variants on graphs and formulate and compare their natural lifts to higher-order domains [3, 7, 12, 13]. In that sense, the present text is not a detached survey, but a foundational bridge between the general higher-order message-passing literature and the more specific question of which propagation geometries are actually appropriate for higher-order structures [3, 7].

1.2 What this document covers

We focus on:

- **Probability and Markov-chain foundations:** the minimum needed to speak precisely about random walks, transition kernels, stationary distributions, reversibility, and mixing.
- **Standard random walk variants on graphs:** simple random walks, lazy walks, weighted and biased walks, restart/PageRank-type walks, multi-step diffusions, continuous-time walks, and non-backtracking walks.
- **Natural higher-order lifts:** how these graph-level variants can be transferred to hypergraphs and, more schematically, to simplicial, cellular, and combinatorial-complex settings.
- **Connection to higher-order message passing:** how random walks become propagation operators and why the walk geometry matters for locality, oversmoothing, oversquashing, and role-sensitive computation.

This selection of topics mirrors the operator-centric perspective already emphasized in surveys of topological and higher-order deep learning, where propagation is viewed as a design choice rather than a downstream implementation detail [3, 7, 12]. We do not carry out implementation or experiments here. The goal is conceptual and mathematical: to prepare a solid theoretical basis for the next stage of the project.

1.3 Primary references and guiding question

We build around the surveys/framework papers Besta et al. [3], Hajij et al. [8], Papillon et al. [12], and the relational-structure viewpoint of Taha et al. [13]. For hypergraph random walks we rely on Carletti et al. [4], and for role-based motivation and hypergraph-sensitive neighborhood constructions we use Eidi and Otter [5]. Background on standard GNNs follows Gilmer et al. [6], Hamilton et al. [9], Kipf and Welling [10], Maron et al. [11], Wu et al. [15], Xu et al. [16].

Concrete question. For hypergraphs, the guiding question for the next stage of the project is whether the constructions of Eidi and Otter [5] can support a genuinely hypergraph-native message passing paradigm defined directly from hyperedge connectivity patterns, rather than through relational graphs or relational structures of the kind considered by Taha et al. [13].

The role of the present survey is to prepare that question at the operator level: by reviewing standard random walk variants on graphs and formulating their natural lifts to higher-order domains, we identify candidate intrinsic propagation geometries that can later be compared with relationalized alternatives.

2 Probability and Markov-chain preliminaries

2.1 Probability spaces, random variables, and stochastic processes

Before defining random walks and Markov chains, it is useful to fix the basic probabilistic language in which they are formulated [2].

Definition 2.1 (Probability space). *A probability space is a triple $(\Omega, \mathcal{F}, \mathbb{P})$, where:*

- Ω is the sample space.
- $\mathcal{F}$ is a σ -algebra of events.
- $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ is a probability measure such that $\mathbb{P}(\Omega) = 1$.

Definition 2.2 (Random variable). *Let (E, ξ) be a measurable space. A random variable on $(\Omega, \mathcal{F}, \mathbb{P})$ with values in E is a measurable map*

$$X : (\Omega, \mathcal{F}) \rightarrow (E, \xi).$$

In the present survey, the state space E will usually be finite or countable.

Definition 2.3 (Stochastic process). *Let T be an index set (typically interpreted as time) and let (E, ξ) be a measurable state space. A stochastic process is a family of random variables*

$$\{X_t\}_{t \in T}, \quad X_t : (\Omega, \mathcal{F}) \rightarrow (E, \xi) \text{ for each } t \in T.$$

Equivalently, a stochastic process is a collection of random variables indexed by a parameter t , which is usually time but may also represent space or another ordering parameter.

Remark 2.1. *Intuitively, a stochastic process is a random system that evolves. For each fixed time t , the variable X_t tells us the random state of the system at that time. For each fixed outcome $\omega \in \Omega$, the map*

$$t \mapsto X_t(\omega)$$

is called a sample path or trajectory of the process.

Definition 2.4 (Discrete-time and continuous-time processes). *A stochastic process is called:*

- *Discrete-time if the index set is $T = \mathbb{N}$, $\mathbb{N} \cup \{0\}$, or another countable ordered set.*
- *Continuous-time if the index set is an interval of $\mathbb{R}$, typically $\mathbb{R}_+$ or $[0, \infty)$.*

In this survey we will mainly work with discrete-time processes.

Definition 2.5 (State space). *The set E in which the variables X_t take values is called the state space of the process. In graph random walks, the state space is often the set of vertices; in higher-order settings, it may instead be a set of hyperedges, incidences, simplices, or other combinatorial carriers.*

Example 2.1 (Random walk). *If X_n denotes the position of a walker after n steps on a graph, then $\{X_n\}_{n \geq 0}$ is a discrete-time stochastic process with state space equal to the vertex set of the graph.*

2.2 Finite-state distributions and stochastic matrices

Since our main objects will be random walks on finite or countable combinatorial structures, it is convenient to make two further definitions explicit.

Definition 2.6 (Distribution on a finite or countable state space). *Let S be a finite or countable set. A probability distribution on S is a family*

$$\mu = (\mu_x)_{x \in S}$$

such that

$$\mu_x \geq 0 \quad \forall x \in S, \quad \sum_{x \in S} \mu_x = 1.$$

If X is an S -valued random variable, its law is the distribution

$$\mu_x = \mathbb{P}(X = x), \quad x \in S.$$

Definition 2.7 (Stochastic matrix). *Let S be a finite or countable state space. A matrix*

$$P = (P_{xy})_{x,y \in S}$$

is called stochastic if

$$P_{xy} \geq 0 \quad \forall x, y \in S, \quad \sum_{y \in S} P_{xy} = 1 \quad \forall x \in S.$$

Thus, each row of P is itself a probability distribution.

Remark 2.2. *With the present convention, P is a row-stochastic matrix: each row of P is a probability distribution, since*

$$P_{xy} \geq 0 \quad \text{and} \quad \sum_{y \in S} P_{xy} = 1 \quad \forall x \in S.$$

In general, nothing is assumed about the column sums. Only in the special case where

$$\sum_{x \in S} P_{xy} = 1 \quad \forall y \in S,$$

the matrix is said to be doubly stochastic. In particular, a doubly stochastic matrix must be square, since the total sum of its entries is both the number of rows and the number of columns.

Remark 2.3. *In the context of random walks, P_{xy} will represent the probability of moving from state x to state y in one step.*

It is also important to distinguish dependence on the current state from dependence on time. The transition matrix P is fixed in the time-homogeneous setting considered here, so it does not depend on t . However, its entries do depend on the current state: P_{xy} is the probability of moving to y given that the chain is currently at x . Equivalently, each row x of P specifies the one-step transition law from state x .

2.3 Finite-state Markov chains

Definition 2.8 (Time-homogeneous Markov chain). *Let S be a finite state space. A discrete-time stochastic process $(X_t)_{t \geq 0}$ with values in S is called a (time-homogeneous) Markov chain if*

$$\mathbb{P}(X_{t+1} = y \mid X_t = x, X_{t-1}, \dots, X_0) = \mathbb{P}(X_{t+1} = y \mid X_t = x)$$

for all $t \geq 0$ and all $x, y \in S$, and if this conditional probability does not depend on t .

Remark 2.4. *The Markov property means that, once the present state is known, the past no longer provides additional information for predicting the next step. In this sense, a Markov chain is a stochastic process with a “memoryless” one-step evolution rule.*

The transition probabilities define a stochastic matrix $P = (P_{xy})_{x,y \in S}$ by

$$P_{xy} := \mathbb{P}(X_{t+1} = y \mid X_t = x).$$

If μ_t denotes the row-vector law of X_t , then

$$\mu_{t+1} = \mu_t P, \quad \mu_t = \mu_0 P^t.$$

Definition 2.9 (Stationary distribution). *A probability vector π on S is stationary for P if*

$$\pi P = \pi.$$

Definition 2.10 (Irreducibility). *A finite Markov chain with transition matrix P is called irreducible if, for every pair of states $x, y \in S$, there exists some $n \geq 1$ such that*

$$(P^n)_{xy} > 0.$$

Equivalently, starting from any state, one can reach any other state in finitely many steps with positive probability.

Remark 2.5. *Intuitively, irreducibility means that the chain is “all connected”: there are no isolated regions of the state space that cannot communicate with each other.*

Definition 2.11 (Aperiodicity). *Let $x \in S$. The period of x is*

$$d(x) := \gcd\{n \geq 1 : (P^n)_{xx} > 0\}.$$

A state x is called aperiodic if $d(x) = 1$. A finite Markov chain is called aperiodic if every state is aperiodic.

Remark 2.6. *The idea here is subtler than irreducibility. A state has periodicity if one can return to it only at times following a rigid pattern. Intuitively, the chain is aperiodic if it is not forced to return only at multiples of some fixed integer greater than 1.*

Remark 2.7. *For a finite irreducible and aperiodic Markov chain, the stationary distribution is unique and the law converges to it from every initial condition:*

$$\mu_0 P^t \rightarrow \pi \quad \text{as } t \rightarrow \infty.$$

Remark 2.8. *A random walk is simply a Markov chain whose states are the objects of a combinatorial structure (for example, the vertices of a graph), and whose transition probabilities are determined by the geometry or incidence pattern of that structure.*

3 Graphs and the basic random walk picture

3.1 Graphs and matrices

Definition 3.1 (Graph). *A finite undirected graph is a pair $G = (V, E)$ where V is a finite vertex set and*

$$E \subseteq \{\{u, v\} : u, v \in V, u \neq v\}.$$

Let $n = |V|$. The adjacency matrix $A \in \mathbb{R}^{n \times n}$ is defined by

$$A_{uv} = \begin{cases} 1, & \{u, v\} \in E, \\ 0, & \text{otherwise.} \end{cases}$$

The degree of u is

$$d_u := \sum_{v \in V} A_{uv},$$

and the degree matrix is

$$D = \text{diag}(d_1, \dots, d_n).$$

Definition 3.2 (Graph Laplacians). *The most common Laplacians are*

$$L = D - A, \quad L_{\text{rw}} = I - D^{-1}A, \quad L_{\text{sym}} = I - D^{-1/2}AD^{-1/2}.$$

These operators are basic both in spectral graph theory and in graph learning, where normalized versions reappear as propagation rules or linear parts of message-passing layers [10, 15].

3.2 Simple random walk

Definition 3.3 (Connected graph). *A graph $G = (V, E)$ is called connected if for every pair of vertices $u, v \in V$, there exists a path from u to v .*

Definition 3.4 (Simple random walk on a graph). *Assume G is connected and has no isolated vertices. The simple random walk on G is the Markov chain on V with transition matrix*

$$P = D^{-1}A, \quad P_{uv} = \frac{A_{uv}}{d_u}.$$

Thus, the walker at node u chooses one of its neighbors uniformly at random. This is the most basic discrete diffusion on a graph and already captures the degree-normalized notion of locality that lies behind many graph propagation operators [10, 15].

Informal intuition. The formula $P = D^{-1}A$ means that each node redistributes one unit of probability mass uniformly across its neighbors. Thus, a high-degree node is not treated as intrinsically more influential at the moment of departure; rather, it spreads its mass more thinly because it has more available directions. This is the basic reason why the walk geometry is degree-normalized rather than based on raw adjacency alone. In long-run terms, however, high-degree nodes are visited more often, since there are more ways to enter them from the surrounding graph. Thus, the simple random walk balances two complementary facts: locally, each node splits mass evenly among its neighbors; globally, nodes with larger degree accumulate more stationary mass.

Definition 3.5 (Reversibility). *A Markov chain with transition matrix P and stationary distribution π is called reversible if*

$$\pi_u P_{uv} = \pi_v P_{vu} \quad \text{for all } u, v \in V.$$

Proposition 3.1 (Reversibility of the simple random walk on an undirected graph). *Let $G = (V, E)$ be a finite, connected, undirected graph, and let*

$$P = D^{-1}A$$

be the transition matrix of the simple random walk. Then the walk is reversible with respect to

$$\pi_u = \frac{d_u}{\sum_{w \in V} d_w}.$$

In particular, π is a stationary distribution.

Proof. For any $u, v \in V$,

$$\pi_u P_{uv} = \frac{d_u}{\sum_{w \in V} d_w} \cdot \frac{A_{uv}}{d_u} = \frac{A_{uv}}{\sum_{w \in V} d_w}.$$

Since G is undirected, its adjacency matrix is symmetric, so $A_{uv} = A_{vu}$. Hence

$$\pi_u P_{uv} = \frac{A_{vu}}{\sum_{w \in V} d_w} = \frac{d_v}{\sum_{w \in V} d_w} \cdot \frac{A_{vu}}{d_v} = \pi_v P_{vu}.$$

Thus the detailed balance condition holds, so the chain is reversible with respect to π . Summing over u , one obtains

$$(\pi P)_v = \sum_{u \in V} \pi_u P_{uv} = \sum_{u \in V} \pi_v P_{vu} = \pi_v \sum_{u \in V} P_{vu} = \pi_v,$$

and therefore π is stationary. □

This already illustrates the main conceptual point: the walk geometry is not raw adjacency, but degree-normalized adjacency. From a learning viewpoint, this normalization is precisely what makes propagation less sensitive to degree heterogeneity than unnormalized adjacency would be [10, 15].

3.3 Random walks, Laplacians, and diffusion

The simple random walk on a graph has transition matrix

$$P = D^{-1}A.$$

Thus, if μ_t denotes the row-vector distribution of the walker at time t , then one discrete step of the walk is given by

$$\mu_{t+1} = \mu_t P.$$

The associated random-walk Laplacian is

$$L_{rw} = I - D^{-1}A = I - P.$$

Equivalently,

$$P = I - L_{rw}.$$

This identity makes the relation between the walk and the Laplacian completely transparent: applying one random-walk step is the same as applying the identity and subtracting the random-walk Laplacian. In this sense, L_{rw} measures how far the one-step propagation rule is from leaving the signal unchanged.

From a diffusion viewpoint, the operator P describes one discrete mixing step, whereas L_{rw} is the corresponding infinitesimal difference operator. In continuous time, the natural diffusion is generated by

$$Q = P - I = -L_{rw},$$

so that the evolution of a distribution μ_t is governed by

$$\frac{d}{dt}\mu_t = \mu_t Q = -\mu_t L_{rw}, \quad \mu_t = \mu_0 e^{-tL_{rw}}.$$

Thus, discrete random walks and continuous-time diffusion are two closely related ways of describing the same propagation geometry on the graph.

This is also why Laplacian-based diffusion operators and graph message-passing propagators look so similar in practice: both are built from the same degree-normalized connectivity structure, and both encode how information is transported and averaged across the graph [6, 10, 15].

In continuous time, diffusion is generated by $-L_{rw}$ or by closely related normalized operators. This is one reason why diffusion operators and message-passing propagators look so similar in practice [6, 10, 15].

3.4 Random walks as propagation rules in GNNs

Many graph neural networks can be written schematically as

$$H^{(\ell+1)} = \sigma\left(P_{\text{prop}}H^{(\ell)}W^{(\ell)}\right),$$

where P_{prop} is some normalized propagation operator [6, 10, 15]. In the simplest case P_{prop} is close to a random-walk step; in more elaborate cases it is a symmetrized, attenuated, restarted, or polynomially filtered version of one. Hence reviewing random walks is not tangential to graph learning: it is directly about the geometry that a message-passing model imposes. This operator view is also what makes the move to higher-order domains conceptually clean [3, 7].

4 Standard random walk variants on graphs

4.1 Lazy random walks

Definition 4.1 (Lazy random walk). *For $\alpha \in (0, 1)$, the lazy version of a graph random walk with kernel¹ P is*

$$P_{\text{lazy}} = \alpha I + (1 - \alpha)P.$$

Thus, with probability α , the walker stays put, and with probability $1 - \alpha$ it makes an ordinary step.

¹Here kernel simply means the one-step transition matrix.

Remark 4.1. Laziness is mainly a device to break periodicity and stabilize the dynamics. For instance, on bipartite graphs the simple random walk has period 2; the lazy walk removes this oscillation. In propagation terms, laziness interpolates between persistence and transport. This kind of stabilization is conceptually close to the addition of self-loops in standard GNN propagators [10, 15].

4.2 Weighted and biased walks

Definition 4.2 (Weighted graph walk). Let $W \in \mathbb{R}^{n \times n}$ be a symmetric nonnegative weight matrix with $W_{uv} = 0$ whenever $\{u, v\} \notin E$. Set

$$s_u := \sum_v W_{uv}, \quad D_W := \text{diag}(s_1, \dots, s_n).$$

The weighted random walk is

$$P_W = D_W^{-1}W, \quad (P_W)_{uv} = \frac{W_{uv}}{s_u}.$$

One may also incorporate node-dependent or edge-dependent biases, writing more generally

$$P_{uv} \propto A_{uv} b(u, v),$$

where $b(u, v) \geq 0$ is some bias factor. Such choices appear whenever one wants to favor hubs, penalize return, or encode affinities stronger than bare adjacency. In machine learning language, this is closely related to replacing uniform aggregation weights by data-dependent or structure-dependent propagation weights [9, 14, 15].

4.3 Random walk with restart and PageRank

Definition 4.3 (Random walk with restart). Given a base transition matrix P , a restart parameter $\gamma \in (0, 1)$, and a restart distribution r , define

$$P_{\text{restart}} = (1 - \gamma)P + \gamma \mathbb{1}r^\top.$$

Thus, at each step the walker either follows the graph with probability $1 - \gamma$ or teleports according to r with probability γ .

A closely related object is personalized PageRank, which is the canonical geometric multi-step extension of a random walk with restart: it aggregates walks of all lengths, but discounts longer trajectories exponentially, thereby balancing global diffusion with retention of source locality.:

$$K_{\text{ppr}}(\alpha) = (1 - \alpha) \sum_{k \geq 0} \alpha^k P^k = (1 - \alpha)(I - \alpha P)^{-1}, \quad 0 < \alpha < 1,$$

whenever the inverse is well-defined.

Remark 4.2. Restart changes the propagation geometry in two ways. First, it regularizes the long-time behavior by preventing over-concentration in bottlenecks. Second, it interpolates between local diffusion and global accessibility. For message passing, this is particularly attractive because it gives a principled way to balance locality and globality. This same tension between local aggregation and longer-range communication is central in oversquashing discussions [1, 13].

4.4 Multi-step kernels and diffusion polynomials

Rather than using a one-step kernel P , one may propagate with a polynomial or convex mixture of powers:

$$K = \sum_{k=0}^{K_{\max}} \beta_k P^k, \quad \beta_k \geq 0, \quad \sum_{k=0}^{K_{\max}} \beta_k = 1.$$

This covers:

- $K = P^m$: exactly m -step information.
- Geometric mixtures: personalized PageRank.
- Truncated heat-type filters.
- General diffusion polynomials used as message-passing propagators.

Remark 4.3. *A multi-step kernel is not merely “more global”. It explicitly chooses which path lengths matter and therefore which structural scales are emphasized. This is important in graph learning because the receptive field and the nature of over-smoothing depend not only on depth, but also on which powers of the propagation operator are actually emphasized [1, 15].*

4.5 Continuous-time random walks

Definition 4.4 (Continuous-time random walk). *A continuous-time random walk on a finite state space with generator Q satisfies*

$$\frac{d}{dt} \mu_t = \mu_t Q, \quad \mu_t = \mu_0 e^{tQ}.$$

On a graph, a canonical choice is

$$Q = P - I = -L_{\text{rw}}.$$

This is the continuous-time counterpart of the simple random walk. In applications, continuous-time diffusion is often algebraically convenient because it leads naturally to matrix exponentials such as e^{-tL} or $e^{-tL_{\text{rw}}}$. In the context of graph learning, this continuous-time viewpoint helps explain why heat-kernel and Laplacian-based propagators arise so naturally [7, 15].

Definition 4.5 (Heat kernel / heat diffusion operator). *Let L be a graph Laplacian (for example, L , L_{rw} , or L_{sym}). For $t \geq 0$, the associated heat kernel or heat diffusion operator is*

$$H_t := e^{-tL}.$$

It is the operator that governs continuous-time diffusion on the graph, in the sense that if a signal or distribution x_t evolves according to

$$\frac{d}{dt} x_t = -L x_t,$$

then

$$x_t = e^{-tL} x_0 = H_t x_0.$$

Remark 4.4. *The heat kernel is the graph analogue of classical heat diffusion: it spreads mass or signal smoothly across the graph as time increases. For small t , the diffusion remains local; for larger t , information is propagated more broadly across the graph. In practice, one often uses truncated or approximated versions of e^{-tL} , which is what is meant here by heat-type filters.*

4.6 Non-backtracking walks

Definition 4.6 (Non-backtracking walk). *A non-backtracking walk forbids the immediate reversal pattern*

$$x_{t-1} \rightarrow x_t \rightarrow x_{t-1},$$

this is, the next step may not instantly undo the previous edge traversal.

To define it as a Markov chain, one enlarges the state space to directed edges or ordered adjacent pairs (u, v) . Then one may move from (u, v) to (v, w) only if $w \sim v$ and $w \neq u$.

Remark 4.5. *Non-backtracking walks often provide a better notion of exploration than ordinary walks on sparse graphs because they reduce trivial bounce-back effects. This matters for walk-based neighborhood graphs, spectral methods, and role-sensitive constructions. More broadly, it is an example of how memory in the walk can change the induced propagation geometry without changing the underlying graph itself [5].*

4.7 Path-based versus walk-based neighborhoods

Definition 4.7 (Walk and path). *Let $G = (V, E)$ be a graph. A walk of length k from u to v is a finite sequence of vertices*

$$v_0, v_1, \dots, v_k$$

such that $v_0 = u$, $v_k = v$, and each consecutive pair (v_{i-1}, v_i) is adjacent for $i = 1, \dots, k$.

A path of length k from u to v is a walk of length k in which no vertex is repeated.

Remark 4.6. *Every path is a walk, but not every walk is a path, since walks may revisit vertices and edges. This distinction becomes important when comparing path-based and walk-based neighborhood notions, and when discussing backtracking effects.*

It is useful to distinguish two derived notions of neighborhood:

- **Path-based neighborhoods:** u and v are related if there exists a path of length k between them.
- **Walk-based neighborhoods:** u and v are related if there exists a walk of length k between them, possibly with repetitions.

The second notion is more diffusion-like and more naturally tied to matrix powers, but it may overcount backtracking effects. Non-backtracking variants sit in between: richer than simple paths, but less degenerate than arbitrary walks. This distinction is already relevant for graph-based role notions and becomes even more consequential in higher-order settings [5, 13].

5 Higher-order domains and the operator viewpoint

5.1 Hypergraphs

Definition 5.1 (Hypergraph). *A hypergraph is a pair $H = (V, \mathcal{E})$, where V is a finite vertex set and*

$$\mathcal{E} \subseteq \mathcal{P}(V) \setminus \{\emptyset\}$$

is a set of hyperedges, each hyperedge being a subset of vertices.

Unlike ordinary graphs, hyperedges may contain more than two vertices. A hypergraph therefore models polyadic interactions natively, which is precisely why it cannot in general be replaced without loss by a simple graph [3, 4, 7].

Definition 5.2 (Incidence matrix). *Let $|V| = n$ and $|\mathcal{E}| = m$. The incidence matrix $B \in \{0, 1\}^{n \times m}$ is defined by*

$$B_{v,e} = \begin{cases} 1, & v \in e, \\ 0, & v \notin e. \end{cases}$$

There are several notions of degree in a hypergraph. At minimum, one distinguishes

$$d(v) := |\{e \in \mathcal{E} : v \in e\}|, \quad |e| := \text{cardinality of the hyperedge } e.$$

These different quantities already signal that there is no unique canonical higher-order normalization, unlike the graph case where degree normalization is comparatively straightforward [4].

5.2 Simplicial complexes

Definition 5.3 (Simplicial complex). *A simplicial complex K on a finite vertex set V is a family of nonempty subsets of V such that:*

- $\{v\} \in K$ for all $v \in V$,
- if $\sigma \in K$ and $\emptyset \neq \tau \subseteq \sigma$, then $\tau \in K$.

Thus, simplicial complexes impose closure under faces. This is the key structural difference from hypergraphs and is what makes boundary and co-boundary operators canonical on simplicial domains [7, 13].

5.3 Cellular and combinatorial complexes

For the present survey, it suffices to keep the following high-level distinction in mind:

- **Cellular complexes** generalize simplicial complexes by allowing more general cells than simplices.
- **Combinatorial complexes** provide a flexible rank-based framework with multiple neighborhood functions and a modular notion of higher-order propagation.

This flexibility is one of the reasons combinatorial-complex-based frameworks, especially CCNNs and GCCNs, have become appealing as unifying languages for topological deep learning [7, 8, 12].

5.4 Random walks as propagation operators

A message-passing layer often has the schematic form

$$H^{(\ell+1)} = \sigma\left(KH^{(\ell)}W^{(\ell)}\right),$$

where K is the propagation operator. From the present viewpoint, K may be:

- A one-step random walk kernel,
- A lazy or biased walk,
- A restart kernel,
- A multi-step diffusion,
- A higher-order lift of any of the above.

So the study of random walks is also the study of the linear geometry that a message-passing network will inherit. This operator reading is explicit in graph MPNNs and remains valid, in a more structured form, on higher-order domains [3, 6, 7].

6 A general template for hypergraph random walks

6.1 Node-hyperedge-node dynamics

The most natural hypergraph walk is an alternating two-stage process:

1. From the current node v , choose an incident hyperedge $e \ni v$,
2. Inside e , choose a destination node $u \in e$.

This perspective is already implicit in microscopic hypergraph diffusion models and is particularly natural from the incidence viewpoint used in hypergraph message passing [4, 7].

Definition 6.1 (General node-hyperedge-node kernel). *Let $H = (V, \mathcal{E})$ be a hypergraph. For each node v , let*

$$\mathcal{E}(v) := \{e \in \mathcal{E} : v \in e\}.$$

Choose:

- A hyperedge-selection law $\alpha_v(e)$ on $\mathcal{E}(v)$,
- A within-hyperedge destination law $\beta_{e,v}(u)$ on e .

Then the induced node-level transition kernel is

$$P_{vu} = \sum_{e \in \mathcal{E}(v)} \alpha_v(e) \beta_{e,v}(u).$$

Remark 6.1 (Informal interpretation). *In this picture, $\mathcal{E}(v)$ is the set of hyperedges available from v , $\alpha_v(e)$ specifies how likely the walker is to choose the hyperedge e from that node, and $\beta_{e,v}(u)$ specifies how likely it is to select u once the hyperedge e has been chosen. The resulting transition probability from v to u is therefore obtained by summing over all possible intermediate hyperedges through which this move can occur:*

$$P_{vu} = \sum_{e \in \mathcal{E}(v)} \alpha_v(e) \beta_{e,v}(u).$$

So the formula is nothing more than the law of total probability applied to the intermediate hyperedge choice. Different hypergraph random walks differ precisely in how they choose the hyperedge-selection law $\alpha_v(e)$ and the within-hyperedge destination law $\beta_{e,v}(u)$.

Remark 6.2. *This formula is the basic unifying device for hypergraph lifts. Different higher-order random walks differ precisely in how they choose $\alpha_v(e)$ and $\beta_{e,v}(u)$. That is the sense in which there is no unique canonical hypergraph random walk, only a design space of hyperedge-mediated geometries [4, 5].*

6.2 Self-loops versus non-lazy conventions

There are two common conventions:

- **Lazy within-hyperedge choice:** allow $u = v$, so $\beta_{e,v}$ is supported on all of e .
- **Non-lazy within-hyperedge choice:** forbid $u = v$, so $\beta_{e,v}$ is supported on $e \setminus \{v\}$.

In this survey, when discussing movement we usually adopt the non-lazy convention, because it corresponds more directly to an actual change of node. Still, the distinction is not cosmetic: allowing or forbidding within-hyperedge self-retention changes both the local kernel and the resulting diffusion behavior [4].

7 Natural hypergraph lifts of graph random walk variants

7.1 Projected random walk

A first lift is not genuinely hypergraph-native: project the hypergraph to a weighted graph, then run an ordinary graph walk.

Definition 7.1 (Projected graph). *Given a hypergraph $H = (V, \mathcal{E})$, its projected weighted graph has the same vertex set V and weighted adjacency*

$$W_{uv} = \sum_{\substack{e \in \mathcal{E} \\ u, v \in e, u \neq v}} \omega_e$$

for some hyperedge weights $\omega_e \geq 0$.

The corresponding projected random walk is simply

$$P^{\text{proj}} = D_W^{-1} W.$$

Remark 7.1. *The projected walk is computationally simple and reuses ordinary graph theory, but it collapses polyadic structure into pairwise couplings. Distinct hypergraphs may induce the same projection. This trade-off between reuse of graph machinery and loss of higher-order information is a recurrent theme in hypergraph learning [3, 7].*

7.2 Uniform-incidence higher-order walk

A more intrinsic choice is: from the current node, choose one incident hyperedge uniformly; then choose another node inside that hyperedge uniformly.

Definition 7.2 (Uniform-incidence higher-order walk). *Assume the non-lazy convention. Define*

$$\alpha_v(e) = \frac{1}{d(v)} \quad \text{for } e \in \mathcal{E}(v),$$

and

$$\beta_{e,v}(u) = \begin{cases} \frac{1}{|e| - 1}, & u \in e \setminus \{v\}, \\ 0, & u = v, \end{cases}$$

because of the non-lazy convention. Then, by Definition 6.1

$$P_{vu}^{\text{HO}} = \sum_{e \ni v, u} \frac{1}{d(v)} \cdot \frac{1}{|e| - 1}.$$

This walk treats each incident hyperedge equally from the current node's perspective. Because the within-hyperedge normalization uses $|e| - 1$, transitions through large hyperedges are diluted across many possible destinations. This is one of the cleanest hypergraph-native analogues of a degree-normalized graph walk [4].

7.3 Size-biased hyperedge selection

Another natural family favors large hyperedges at the first stage.

Definition 7.3 (Size-biased hypergraph walk). *Fix a nonnegative function $g : \mathbb{N}_{\geq 2} \rightarrow \mathbb{R}_{\geq 0}$. Define*

$$\alpha_v^{(g)}(e) = \frac{g(|e|)}{\sum_{f \in \mathcal{E}(v)} g(|f|)},$$

and keep the same within-hyperedge destination rule

$$\beta_{e,v}(u) = \begin{cases} \frac{1}{|e| - 1}, & u \in e \setminus \{v\}, \\ 0, & u = v. \end{cases}$$

Then

$$P_{vu}^{(g)} = \sum_{e \ni v, u} \frac{g(|e|)}{\sum_{f \in \mathcal{E}(v)} g(|f|)} \cdot \frac{1}{|e| - 1}.$$

Special cases include:

- $g(k) \equiv 1$: uniform-incidence walk.
- $g(k) = k$ or $g(k) = k - 1$: size-biased walk.
- $g(k) = k^\gamma$: nonlinear size bias.

Remark 7.2. *This lift is especially relevant because it replaces a purely one-step hypergraph walk by a geometrically weighted mixture of walks of all lengths. In this way, it turns a local one-step hypergraph geometry into a multiscale diffusion geometry: short trajectories still dominate, but longer-range higher-order connectivity also becomes visible. The parameter η therefore provides a*

principled way to interpolate between immediate higher-order locality and broader structural accessibility.

From the perspective of the project, such kernels are particularly interesting because they provide natural intrinsic propagation operators once a base hypergraph walk P_H has been fixed. In that sense, they are plausible candidates for role-sensitive higher-order message passing, and they are directly relevant to the broader research question of whether a genuinely hypergraph-native message passing paradigm can be defined from hyperedge connectivity patterns rather than via auxiliary relational graphs or relational structures [5, 13].

7.4 Microscopic versus projected viewpoints

It is important to distinguish two related but conceptually different constructions:

1. **Microscopic node-hyperedge-node walk:** the hypergraph is traversed explicitly through incidence.
2. **Equivalent weighted graph viewpoint:** under suitable choices of weights, the induced node-level chain may be represented as a walk on a weighted projected graph.

This equivalence can be algebraically useful, but it should not obscure the conceptual point: the higher-order model is still defined by a choice of hyperedge-mediated geometry. This distinction also mirrors the broader contrast between intrinsic higher-order propagation and graphification/relationalization [3, 7, 13].

7.5 Lazy hypergraph lifts

Given any node-level hypergraph kernel P_H , define its lazy version by

$$P_{H,\text{lazy}} = \alpha I + (1 - \alpha)P_H.$$

As on graphs, laziness is useful to avoid periodic artifacts and to interpolate between persistence and movement. Importantly, however, here the persistence is built on top of an already nontrivial higher-order local geometry [4].

7.6 Restart and personalized higher-order diffusion

Given a hypergraph walk kernel P_H , its restart version is

$$P_{H,\text{restart}} = (1 - \gamma)P_H + \gamma \mathbb{1}r^\top,$$

where r is a restart distribution on vertices.

Similarly, the corresponding higher-order personalized PageRank kernel is

$$K_{H,\text{ppr}}(\eta) = (1 - \eta) \sum_{k \geq 0} \eta^k P_H^k = (1 - \eta)(I - \eta P_H)^{-1}.$$

Remark 7.3. *This is one of the most important natural lifts for the later research agenda: it turns a one-step hypergraph geometry into a multi-scale diffusion geometry and gives a principled way to interpolate between local higher-order structure and global accessibility. From the point of view of the project, these are especially plausible candidates for role-sensitive propagation operators [5, 13].*

7.7 Multi-step hypergraph kernels

Once a base hypergraph walk P_H has been fixed, one may form polynomial or mixture kernels exactly as on graphs:

$$K_H = \sum_{k=0}^{K_{\max}} \beta_k P_H^k.$$

Conceptually, the algebra does not change. What changes is the meaning of a step: the one-step geometry itself is already hyperedge-mediated. This is why the same formal expression may correspond to genuinely different propagation semantics on graphs and hypergraphs [4, 7].

7.8 Non-backtracking hypergraph lifts

A non-backtracking lift on a hypergraph should prevent immediate reversal of the incidence decision. There are several reasonable formulations:

- **Hyperedge-memory formulation:** remember the previously used hyperedge and forbid selecting it again immediately.
- **Node-memory formulation:** remember the previous node and forbid immediate return to it.
- **Incidence-state formulation:** enlarge the state space to pairs (v, e) with $v \in e$, then define a Markov chain on incidence states.

Definition 7.4 (Incidence-state non-backtracking walk, schematic). *Let the state space be*

$$S = \{(v, e) : v \in e\}.$$

A transition

$$(v, e) \rightarrow (u, f)$$

is allowed if $u \in e \cap f$, $u \neq v$, and f is not an immediate reversal of the previously used incidence choice.

Remark 7.4. *The precise choice of what counts as a backtrack is part of the model design. This is much less canonical than on ordinary graphs, and that is precisely why hypergraph walk design is an open and interesting research direction. It is also why one should be cautious about speaking of the hypergraph non-backtracking walk as though there were a unique standard object [4, 5].*

8 Natural lifts beyond hypergraphs

8.1 Simplicial complexes: up, down, and same-rank walks

On a simplicial complex, the walk need not remain on vertices. One may define random walks on:

- Vertices (rank 0),
- Edges (rank 1),

- Higher-dimensional simplices,
- Across several ranks simultaneously.

The key geometric relations are:

- **Boundary/downward relation:** move from a simplex to one of its faces.
- **Co-boundary/upward relation:** move from a simplex to one of its cofaces.
- **Same-rank lower adjacency:** simplices sharing a face.
- **Same-rank upper adjacency:** simplices sharing a coface.

A random-walk lift on simplices is therefore specified by:

1. The carrier rank(s),
2. The incidence relation(s) one is allowed to traverse,
3. The normalization/weighting of those traversals.

These distinctions align closely with the different local channels used in simplicial message passing and in the relational-structure treatment of topological propagation [7, 13].

8.2 Edge-based walks and Hodge operators

A simplicial complex supports diffusion not only on vertices, but also on higher-rank carriers such as edges, triangles, and higher-dimensional simplices. This is one of the main differences from ordinary graph random walks, where the state space is typically the vertex set. In the simplicial setting, the boundary and co-boundary relations induce incidence operators and, from them, rank-specific Laplacians, usually called Hodge Laplacians [7, 13].

From this viewpoint, the normalized Hodge 1-Laplacian plays for edge-supported signals the same structural role that the graph Laplacian plays for vertex-supported signals on an ordinary graph. The analogy is therefore:

$$\text{graph Laplacian} \longleftrightarrow \text{vertex walk}, \quad \text{Hodge 1-Laplacian} \longleftrightarrow \text{edge walk}.$$

The point is not that one simply renames vertices as edges, but that the carrier of the diffusion has changed. A signal now lives on 1-simplices rather than on vertices, and the corresponding diffusion operator propagates information through edge-level adjacency and incidence structure. In this sense, an edge-based walk is the natural higher-order analogue of an ordinary random walk once the relevant carrier is the set of edges rather than the set of vertices.

This makes explicit that higher-order diffusion can live on non-vertex carriers and can interact with topology in ways unavailable to ordinary node-based walks. In particular, it shows that a lift from graphs to simplicial complexes is not only a matter of changing the domain, but also of changing where the stochastic mass or signal is allowed to live [7].

Remark 8.1. *From the present survey’s viewpoint, Hodge-based walks are natural higher-order lifts of the graph random-walk idea because they transport the same diffusion logic to cochain spaces and higher-rank carriers.*

8.3 Across-dimension walks

An even richer lift allows transitions between simplices of different dimensions. Schematically, one may have a block operator acting on

$$\bigoplus_{k=0}^d C^k,$$

where C^k is the space of rank- k signals. Then transitions may move

$$\text{vertex} \leftrightarrow \text{edge} \leftrightarrow \text{triangle} \leftrightarrow \dots$$

according to incidence-based stochastic rules. This is the higher-order analogue of allowing several carrier types to exchange mass, and it resonates with multi-rank propagation frameworks in topological deep learning [7, 8].

8.4 Combinatorial complexes and multi-neighborhood lifts

In a combinatorial-complex framework, one typically has several neighborhood functions. A natural random-walk interpretation is therefore:

1. Choose a neighborhood channel.
2. Move according to the stochastic kernel associated with that channel.
3. Optionally mix several channels by convex combination.

Thus one obtains

$$K = \sum_{r=1}^R \lambda_r P_r, \quad \lambda_r \geq 0, \quad \sum_{r=1}^R \lambda_r = 1,$$

where each P_r is a walk on one neighborhood graph or one Hasse-type expansion.

Remark 8.2. *This is the probabilistic version of the modular multi-neighborhood decompositions used in generalized combinatorial complex neural networks. In that sense, random-walk language provides a natural interpretation of why GCCN-style frameworks can combine several propagation geometries coherently [7, 12].*

9 Comparing walk variants: what actually changes?

9.1 State space

Two walks may live on very different carriers:

$$V, \quad E, \quad \{(v, e) : v \in e\}, \quad \text{simplices of rank } k, \quad \text{multi-rank states.}$$

A change in carrier is already a major geometric change. This is why “lifting a walk” is not just a matter of changing formulas: one is also changing where the stochastic mass is allowed to live [3, 7, 8].

9.2 Neighborhood notion

The relevant adjacency may mean:

- Ordinary graph adjacency.
- Co-membership in a hyperedge.
- Shared face or shared coface.
- Relation in an auxiliary graph.
- Relation in a walk-induced neighborhood graph.

This is central to later role-based questions: different neighborhood notions produce different notions of local similarity. The point is especially important in hypergraphs, where distinct neighborhood constructions may preserve or destroy role information in different ways [5, 13].

9.3 Bias and stationary distribution

Every normalization or weighting changes what the walk regards as structurally important.

- Degree-normalized walks favor high-degree regions.
- Size-biased hypergraph walks emphasize large group interactions.
- Uniform-incidence walks prevent large hyperedges from dominating the first-stage selection.
- Restart-based kernels prevent excessive concentration in traps and bottlenecks.
- Non-backtracking variants reward exploration rather than immediate reversal.

So a walk choice is simultaneously a geometric bias choice. In a learning setting, it becomes a modeling choice about which parts of the structure should be easiest to reach and easiest to average over [4, 5, 15].

9.4 Query complexity and locality

For large hypergraphs, there is also a computational distinction.

- A projected walk may require access to many pairwise co-memberships.
- An incidence-based walk may require only sampling an incident hyperedge and then sampling within it.
- Non-backtracking variants enlarge the state space but may improve exploration efficiency.

So the comparison is not only theoretical but also algorithmic. This computational angle is one reason why auxiliary-graph methods remain attractive even when they are conceptually lossy [3, 12].

9.5 One-step versus multi-step geometry

A one-step kernel defines the local geometry. A multi-step kernel defines the geometry visible after controlled diffusion. These are not the same question. In particular,

$$P_H \quad \text{and} \quad (I - \eta P_H)^{-1}$$

encode genuinely different propagation regimes, even though the second is built from the first. This distinction is especially relevant for oversquashing and role-sensitive tasks, where long-range behavior may matter more than the raw one-step neighborhood [1, 5, 13].

10 Structural roles, equivalence, and why the walk matters

10.1 Structural equivalence versus regular equivalence

The later research direction is motivated more broadly by the question of whether hypergraph propagation should be defined intrinsically from hyperedge connectivity patterns or via auxiliary relational constructions. It is therefore useful to recall the classical distinction.

Definition 10.1 (Structural equivalence, graph intuition). *Two vertices u and v are structurally equivalent if they have exactly the same neighborhood pattern.*

Definition 10.2 (Regular equivalence, graph intuition). *Two vertices u and v are regularly equivalent if they are connected to vertices that play the same roles, even if they are not connected to the same specific vertices.*

Thus structural equivalence is stricter, whereas regular equivalence is role-based. The importance of this distinction for the present project is that many propagation rules are proximity-sensitive but not necessarily role-sensitive [5].

10.2 Neighborhood graphs induced by walks

A walk can induce derived graphs that connect vertices according to length- k paths, walks, or non-backtracking walks. These derived neighborhood graphs influence which vertices appear similar and which structural patterns become visible after propagation.

This is particularly relevant in higher-order settings, where one may define neighborhood graphs from:

- Projected pairwise connectivity.
- Hyperedge co-membership.
- Incidence-state transitions.
- Multi-step hypergraph diffusions.
- Non-backtracking higher-order walks.

Remark 10.1. *The choice of walk is therefore also a choice of similarity geometry. This is exactly why restart, multi-step, and non-backtracking hypergraph lifts are not cosmetic variants: they may alter which structural roles are preserved or highlighted [5].*

10.3 Curvature and transport viewpoint

A further viewpoint, especially relevant for structural comparison, is to attach to each state x its one-step neighborhood measure μ_x and compare these measures using optimal transport. Then a pair of states is judged similar not only by direct adjacency, but by how cheaply one local measure can be transported into the other.

At a high level, this connects

$$\text{random walk} \longrightarrow \text{local probability measure} \longrightarrow \text{diffusion geometry}.$$

In hypergraphs, different walk choices produce different local measures, and therefore different notions of neighborhood overlap or role similarity. This perspective is particularly close in spirit to recent work relating hypergraph geometry, induced neighborhood graphs, and equivalence notions [5].

11 Random walks as propagation operators in higher-order message passing

11.1 Graph case

A standard graph layer may be written as

$$H^{(\ell+1)} = \sigma\left(PH^{(\ell)}W^{(\ell)}\right),$$

or with a diffusion kernel K ,

$$H^{(\ell+1)} = \sigma\left(KH^{(\ell)}W^{(\ell)}\right).$$

Thus:

- $P = D^{-1}A$ gives ordinary one-step mixing.
- P_{lazy} gives stabilized one-step mixing.
- K_{ppr} gives restart-based multiscale diffusion.
- A polynomial in P gives explicit multi-hop propagation.
- A non-backtracking operator gives an exploration-biased propagation rule.

This operator reading is consistent with the standard MPNN perspective and with the normalization patterns of classical GNN architectures such as GCN, GraphSAGE, and attention-based models [6, 9, 10, 14].

11.2 Hypergraph case

Once a base hypergraph walk P_H has been fixed, the same schematic construction becomes

$$H^{(\ell+1)} = \sigma\left(P_H H^{(\ell)} W^{(\ell)}\right),$$

or more generally

$$H^{(\ell+1)} = \sigma(K_H H^{(\ell)} W^{(\ell)}), \quad K_H = \sum_{k=0}^{K_{\max}} \beta_k P_H^k.$$

This is the operator-level bridge between hypergraph random walks and higher-order message passing. The point is not merely that one replaces the graph operator P by a hypergraph operator P_H , but that the choice of P_H already encodes a specific higher-order propagation geometry. Different hypergraph walks correspond to different notions of locality, different normalizations of hyperedge-mediated movement, and therefore different ways of mixing information across the domain.

This is precisely why hypergraph message passing is not determined uniquely by the hypergraph domain itself. Even after the carrier has been fixed, one must still choose the propagation operator: for instance, whether to use a projected walk or an intrinsic node-hyperedge-node walk, whether incident hyperedges should be weighted uniformly or size-biased, and whether the resulting one-step geometry should later be enriched by laziness, restart, multi-step diffusion, or non-backtracking constraints.

From the perspective of the project, this is the key conceptual point. A hypergraph message-passing architecture is not specified only by saying that the data live on a hypergraph; it is also specified by which hypergraph-native walk geometry is used to transport and aggregate information. In that sense, the design of P_H is part of the model design, not merely an implementation detail. It is also the place where the broader tension between intrinsic hypergraph propagation and relationalized or auxiliary-graph constructions becomes operational: different choices of P_H instantiate genuinely different higher-order propagation paradigms.

11.3 Why restart and multi-step lifts are especially relevant

In the graph setting, restart and multi-step diffusion are well-known ways to control locality versus globality. In the hypergraph setting, they become even more interesting because they act *after* a higher-order notion of locality has already been chosen.

This leads to a natural follow-up question for the project: given a hypergraph-native one-step walk, does adding restart or multi-step diffusion improve the model’s ability to capture structural roles, equivalence patterns, or long-range higher-order organization?

This does not replace the broader research question of whether genuinely hypergraph-native propagation should be defined directly from hyperedge connectivity patterns rather than via relationalization. Rather, it identifies a particularly relevant family of intrinsic operators once such a one-step geometry has been fixed.

12 Synthesis and outlook

12.1 A useful list of design knobs

Across the survey, the main design knobs are:

1. **Domain:** graph, hypergraph, simplicial complex, cellular complex, combinatorial complex, or relationalized auxiliary representation.
2. **Carrier:** vertices only, incidences, edges, simplices, or multi-rank states.

3. **Neighborhood notion:** adjacency, co-membership, shared face/coface, auxiliary relation, or walk-induced neighborhood.
4. **One-step geometry:** uniform, weighted, size-biased, edge-dependent, or directional.
5. **Long-range mechanism:** lazy, restart-based, multi-step, heat-type, or non-backtracking.
6. **Operator use:** analysis only, or direct use as a propagation kernel in a learnable model.

This list is useful precisely because it makes explicit that higher-order propagation is not a single design decision but a coordinated choice across several levels of structure [7, 8, 12].

12.2 What counts as a natural lift?

From the viewpoint of this document, a natural lift of a graph random walk variant to a higher-order structure should satisfy two criteria:

1. It should preserve the idea of the graph variant. For example, laziness should remain self-persistence, restart should remain teleportation to a prescribed law, and non-backtracking should still penalize immediate reversal.
2. It should be intrinsic to the higher-order domain whenever possible. That is, the lift should be definable directly from higher-order incidence or rank structure, rather than only through projection.

The second criterion matters because otherwise one risks replacing a higher-order problem by an ordinary graph problem too early and losing the very structural distinctions one wanted to preserve [3, 5, 13].

12.3 Returning to the open question

The survey leads directly to the following operator-level research question: among hypergraph-native walks and their restart/multi-step/non-backtracking lifts, which propagation geometries best capture structural and regular roles, and how do they compare with relationalized or auxiliary-graph alternatives when used inside higher-order message passing networks?

This should be understood as a concrete refinement of the broader guiding question of the project: whether a genuinely hypergraph-native message passing paradigm can be defined directly from hyperedge connectivity patterns rather than via relational graphs or relational structures.

In this sense, the present review is meant to isolate the main intrinsic design choices on the random-walk side before such a comparison is carried out.

13 Conclusion

We have reviewed the standard random walk variants on graphs and organized their natural lifts to higher-order structures around a unified operator template. The main message is not that there exists a single canonical higher-order random walk. On the contrary, the higher-order setting exposes a meaningful design space. On graphs, the designer chooses between simple, lazy, weighted,

restart, multi-step, continuous-time, and non-backtracking walks. On hypergraphs and complexes, one must additionally choose what it even means to be adjacent, which carriers are allowed to hold mass, and how one-step higher-order movement should be normalized [3, 4, 7].

This makes random walks especially relevant to higher-order graph learning. They are not only stochastic processes on combinatorial objects; they are also propagation operators, neighborhood definitions, and diffusion geometries. The next step, therefore, is not merely to use a hypergraph walk, but to understand which higher-order walk geometry is most appropriate for the structural questions and message-passing objectives of interest [5, 13]. More specifically, the broader goal is to determine whether these intrinsic walk geometries can support a genuinely hypergraph-native message passing paradigm defined directly from hyperedge connectivity patterns rather than via relationalized constructions.

Acknowledgements

This draft is prepared as part of a research project supervised by Prof. Guido Montúfar.

References

- [1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.05205.
- [2] Xavier Bardina and Marco Ferrante. *An Excursion into Markov Chains*. Springer, 2023. March 11, 2026.
- [3] Maciej Besta, Florian Scheidl, Lukas Gianinazzi, Grzegorz Kwasniewski, Shachar Klaiman, Jürgen Müller, and Torsten Hoefer. Demystifying higher-order graph neural networks. *arXiv preprint*, 2025. arXiv:2406.12841v4.
- [4] Timoteo Carletti, Federico Battiston, Giulia Cencetti, and Duccio Fanelli. Random walks on hypergraphs. *Physical Review E*, 101(2):022308, 2020. arXiv:1911.06523.
- [5] Marzieh Eidi and Nina Otter. Geometric characterisation of structural and regular equivalences in undirected (hyper)graphs. *arXiv preprint*, 2025. arXiv:2512.24961v1.
- [6] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1704.01212.
- [7] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Nina Miolane, Aldo Guzmán-Sáenz, Karthikeyan Natesan Ramamurthy, Tolga Birdal, Tamal K. Dey, Soham Mukherjee, Shreyas N. Samaga, Neal Livesay, Robin Walters, Paul Rosen, and Michael T. Schaub. Topological deep learning: Going beyond graph data. *arXiv preprint*, 2023. arXiv:2206.00606v3.
- [8] Mustafa Hajij, Lennart Bastian, Sarah Osentoski, Hardik Kabaria, John L. Davenport, Sheik Dawood, Balaji Cherukuri, Joseph G. Kocheemoolayil, Nastaran Shahmansouri, Adrian Lew, Theodore Papamarkou, and Tolga Birdal. Copresheaf topological neural networks: A generalized deep learning framework. *arXiv preprint*, 2025. arXiv:2505.21251v3.

- [9] Will Hamilton, Zhitaoying, and Jure Leskovec. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, pages 1025–1035. Curran Associates, Inc., 2017. ISBN 9781510860964.
- [10] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.
- [11] Haggai Maron, Heli Ben-Hamu, Nadav Shami, and Yaron Lipman. Invariant and equivariant graph networks. In *International Conference on Learning Representations (ICLR)*, 2019. Published as a conference paper at ICLR 2019.
- [12] Mathilde Papillon, Guillermo Bernardez, Claudio Battiloro, and Nina Miolane. Topotune: A framework for generalized combinatorial complex neural networks. *arXiv preprint*, 2025. arXiv:2410.06530v4.
- [13] Diaaeldin Taha, James Chapman, Marzieh Eidi, Karel Devriendt, and Guido Montúfar. Demystifying topological message-passing with relational structures: A case study on oversquashing in simplicial message-passing. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2506.06582v1.
- [14] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.
- [15] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386. arXiv:1901.00596.
- [16] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.

A A compact dictionary of graph and hypergraph operators

For quick reference, we summarize the most important operators appearing in the survey.

- Graph adjacency: A .
- Graph degree matrix: D .
- Simple graph walk: $P = D^{-1}A$.
- Graph random-walk Laplacian: $L_{\text{rw}} = I - P$.
- Lazy graph walk: $P_{\text{lazy}} = \alpha I + (1 - \alpha)P$.
- Restart kernel: $P_{\text{restart}} = (1 - \gamma)P + \gamma \mathbb{1} r^\top$.
- Personalized PageRank kernel: $K_{\text{ppr}}(\eta) = (1 - \eta)(I - \eta P)^{-1}$.
- Hypergraph incidence matrix: B .
- General node-hyperedge-node walk:

$$P_{vu} = \sum_{e \ni v} \alpha_v(e) \beta_{e,v}(u).$$

- Multi-step higher-order diffusion:

$$K_H = \sum_{k=0}^{K_{\max}} \beta_k P_H^k.$$

These operators recur throughout graph learning and higher-order message passing because they are exactly the linear maps through which local geometry is turned into propagation [6, 7, 12].

B Minimal operator-centric template for later use

A useful template for later message-passing experiments is:

$$X^{(\ell+1)} = \sigma \left((\lambda_0 I + \lambda_1 P_H + \lambda_2 P_H^2 + \cdots + \lambda_K P_H^K) X^{(\ell)} W^{(\ell)} \right),$$

with

$$\lambda_k \geq 0, \quad \sum_{k=0}^K \lambda_k = 1.$$

This single expression includes:

- one-step higher-order propagation,
- lazy propagation,
- truncated multi-step diffusion,
- restart-like behavior through appropriate coefficients,
- and a direct bridge to higher-order message passing.

This is the algebraic form in which the later comparison between different walk geometries can be made most cleanly [3, 4, 7].

C Derivation of the symmetric normalized Laplacian

A standard normalization (especially useful in spectral derivations) is obtained by scaling L_{un} on the left and right by $D^{-1/2}$:

$$L_{\text{sym}} := D^{-1/2} L_{\text{un}} D^{-1/2} = D^{-1/2} (D - A) D^{-1/2} = I - D^{-1/2} A D^{-1/2}.$$

This L_{sym} is the (symmetric) normalized Laplacian, often denoted L_{norm} .

Intuition (degree-normalization): The factor $D^{-1/2}$ degree-normalizes node-wise signals: it rescales a vector $x \in \mathbb{R}^n$ to $\tilde{x}$ with $\tilde{x}_i = x_i / \sqrt{d_i}$. This compensates for degree heterogeneity, so high-degree nodes do not dominate simply because they have more neighbors. This same normalization logic is exactly what later reappears in stable graph propagation rules such as GCN-type layers [10, 15].